

SmartDB Communication System

Table of Contents

1. [The big picture — why this stack](#)
2. [Networking basics you need to know](#)
3. [UDP — what it is and why we use it](#)
4. [Basic cryptography concepts](#)
5. [AES-128 encryption](#)
6. [Cipher Block Chaining \(CBC\) mode](#)
7. [Message Authentication Code \(MAC\)](#)
8. [TLS — the security layer most people know](#)
9. [DTLS — TLS adapted for UDP](#)
10. [Pre-Shared Keys \(PSK\)](#)
11. [mbedTLS — the library we actually use](#)
12. [Replay attack prevention](#)
13. [How everything fits together in SmartDB](#)
14. [Glossary](#)

The big picture

SmartDB sends sensor data (current readings, anomaly scores, alerts) from an ESP32 node to a gateway over a WiFi network. This communication needs to be:

- **Fast** — sensor data is sent every 100ms, alerts must arrive quickly
- **Secure** — an attacker who can inject a fake OVERRIDE command could shut down a homeowner's electrical system maliciously, or prevent a real shutdown during a fault
- **Reliable enough** — UDP can drop packets, we need a strategy for that
- **Lightweight** — ESP32 has limited RAM and CPU, we can't run heavy protocols

The protocol stack we chose to meet these requirements:

Application layer	→	SmartDB custom protocol (message types 0x01-0x09)
Security layer	→	DTLS 1.2 (Datagram TLS)
Transport layer	→	UDP (User Datagram Protocol)
Network layer	→	IP
Physical layer	→	WiFi / Ethernet

Each layer has one job. Understanding each layer separately makes the whole system easier to reason about, implement, and debug.

Networking basics

IP addresses and ports

Every device on a network has an **IP address** — a unique identifier like **192.168.1.100**. When a device sends data, the receiving device needs to know which application to give the data to — that's what **ports** are for.

```
IP address = which device
Port number = which application on that device
```

SmartDB uses UDP port **5684** — this is the IANA-registered standard port for DTLS, the same way port 443 is standard for HTTPS.

Packets

Data travels over a network in **packets** — fixed-size chunks of bytes. Every packet has a header (metadata: source, destination, length) and a payload (the actual data). SmartDB's application protocol defines exactly what goes in each payload depending on the message type.

UDP

What UDP is

UDP (User Datagram Protocol) is a transport protocol. It sends packets from one device to another with minimal overhead.

UDP is **connectionless** — there is no handshake before sending, no confirmation that the other side received anything, and no guarantee that packets arrive in order. You fire a packet and forget it.

Why not TCP?

TCP (Transmission Control Protocol) is the alternative — it guarantees delivery and ordering by re-sending lost packets and buffering out-of-order ones. This reliability comes at a cost:

- TCP requires a 3-way handshake before any data flows (adds latency)
- Re-sending lost packets adds delay — if a packet is lost, TCP waits for a retransmission before delivering anything that came after it
- TCP keeps a connection open, which has memory overhead

For SmartDB, these costs matter:

Sensor data every 100ms — a dropped packet is fine, the next one arrives **in** 100ms anyway. We don't **want** to wait for a retransmission.

Alert messages are high priority — we handle their reliability ourselves (retransmission policy) so we can control exactly how urgently they retry.

Emergency override — we need this now, not after TCP's flow control decides it's a good time to send.

UDP gives us raw speed and control. We implement our own reliability where we actually need it (per our retransmission policy), and accept packet loss where it doesn't matter (telemetry).

UDP packet structure

Source port	Dest port	Length	Checksum	Payload
16 bits	16 bits	16 bits	16 bits	variable

The payload is whatever the layer above (DTLS) puts in it.

Basic cryptography

You need to understand four concepts before the security layer makes sense.

Plaintext and ciphertext

Plaintext is readable data — your sensor readings, alert messages. **Ciphertext** is the scrambled version produced by encryption. An attacker who intercepts ciphertext cannot read the original plaintext without the key.

Symmetric vs asymmetric encryption

Symmetric encryption uses the same key to encrypt and decrypt. Both sides must know the key in advance. Fast — suitable for real-time data.

Asymmetric encryption uses a public/private key pair — you encrypt with a public key, decrypt with the paired private key. Used in TLS certificate handshakes. Slower — used to establish session keys, not for bulk data.

SmartDB uses **symmetric encryption (AES-128)** for the actual data, and **PSK (pre-shared key)** instead of asymmetric key exchange to keep things simple and fast on a constrained device.

Keys

A key is a secret value that controls how encryption works. With the same algorithm and the same key, encryption always produces the same ciphertext from the same plaintext. Without the key, the ciphertext is meaningless.

Key size determines how hard it is to guess the key by brute force:

- AES-128 → 128-bit key → 2^{128} possible keys → effectively unbreakable with current and foreseeable computing power

Hash functions and integrity

A **hash function** takes any input and produces a fixed-size output (a "digest"). The same input always produces the same output. A tiny change in input produces a completely different output.

Hash functions are one-way — you cannot reverse a hash to find the input.

Used in SmartDB for: verifying that a packet was not modified in transit (integrity checking via MAC — see section 7).

AES-128 encryption

AES (Advanced Encryption Standard) is the world's most widely used symmetric encryption algorithm. "128" refers to the key size in bits.

How AES works (conceptually)

AES operates on fixed-size **blocks** of 16 bytes (128 bits) at a time. It transforms a 16-byte plaintext block into a 16-byte ciphertext block using a series of mathematical operations (substitution, permutation, mixing) controlled by the key. This process is called a **round**, and AES-128 performs 10 rounds.

The result is a ciphertext that looks completely random to anyone without the key.

What AES guarantees

- **Confidentiality** — without the key, ciphertext reveals nothing about the plaintext
- **Deterministic** — same key + same plaintext always gives same ciphertext

What AES alone does NOT guarantee

- **Integrity** — AES doesn't detect if ciphertext was modified in transit
- **Authenticity** — AES doesn't prove who encrypted the data
- These gaps are filled by MAC (section 7) and DTLS (section 9)

AES on ESP32

ESP32 has a **hardware AES accelerator** — AES operations run in dedicated silicon, not in software on the CPU. This is significantly faster and uses less power than software AES. mbedTLS uses this automatically when configured correctly (see section 11).

CBC mode (Cipher Block Chaining)

AES alone only encrypts one 16-byte block at a time. Real messages are longer than 16 bytes. **Modes of operation** define how AES is applied repeatedly to encrypt messages of arbitrary length.

SmartDB uses **CBC (Cipher Block Chaining)** mode.

The problem CBC solves

If you just encrypt each 16-byte block independently (called ECB mode), identical blocks of plaintext produce identical blocks of ciphertext. An attacker can see patterns in your data even without breaking the key — like seeing the same ciphertext block appear every time a specific sensor reading is sent.

How CBC works

Before encrypting each block, CBC XORs it with the previous ciphertext block. This chains all blocks together — each block's encryption depends on everything that came before it.

```
Block 1: Plaintext1 XOR IV → AES encrypt → Ciphertext1
Block 2: Plaintext2 XOR Ciphertext1 → AES encrypt → Ciphertext2
Block 3: Plaintext3 XOR Ciphertext2 → AES encrypt → Ciphertext3
```

IV (Initialization Vector) — the random value used to start the chain for the first block. Must be different for every message. If the IV is reused, an attacker can detect identical messages. DTLS generates a fresh IV for every record.

CBC's tradeoff

CBC requires the IV (16 bytes) to be sent alongside the ciphertext so the receiver can decrypt. This adds overhead but is worth it for the pattern hiding it provides.

CBC also requires **padding** — if the message isn't a multiple of 16 bytes, it must be padded to the next multiple. mbedTLS handles this automatically.

Why not GCM?

AES-GCM is a more modern mode that combines encryption and authentication in one step, eliminating the need for a separate MAC. It's more efficient and generally preferred in new designs. CBC was chosen for SmartDB's initial design due to wider ESP32/mbedTLS documentation coverage — this is flagged as an open decision and may be revisited.

MAC (Message Authentication Code)

Encryption provides **confidentiality** (nobody can read the data) but not **integrity** (nobody can detect if the data was modified).

A MAC solves this. A MAC is a short fixed-size tag computed from the message and a secret key. The receiver recomputes the MAC and compares — if they match, the message was not modified.

```
Sender:    MAC = HMAC(key, message) → sends message + MAC
Receiver:  recompute MAC from received message
           if computed MAC == received MAC → message is authentic
           if they differ → message was tampered with, reject it
```

HMAC (Hash-based MAC) is the specific MAC algorithm used in DTLS — it applies a hash function (SHA-256 in DTLS 1.2) to the message and key.

In SmartDB's DTLS configuration:

- AES-128-CBC encrypts the payload (confidentiality)
- HMAC-SHA256 authenticates the payload (integrity + authenticity)

Together these are referred to as the **cipher suite**: `TLS_PSK_WITH_AES_128_CBC_SHA256`

TLS

TLS (Transport Layer Security) is the security protocol used by HTTPS, email, and most secure internet communication. It provides:

- **Authentication** — proves the server is who it claims to be
- **Confidentiality** — encrypts the session
- **Integrity** — detects tampering

TLS runs on top of TCP. It performs a **handshake** before any application data flows — this handshake negotiates which cipher suite to use, exchanges keys, and authenticates the parties.

TLS handshake (simplified)

```
Client → Server:  ClientHello (supported cipher suites)
Server → Client:  ServerHello (chosen cipher suite) + Certificate
Client → Server:  Key exchange material + Finished
Server → Client:  Finished
→ Encrypted session begins
```

Why not TLS directly?

TLS requires TCP — it assumes reliable, ordered delivery. UDP doesn't provide this. Running TLS over UDP would break because TLS handshake messages could arrive out of order or get lost with no recovery mechanism. DTLS (section 9) solves this.

DTLS

DTLS (Datagram Transport Layer Security) is TLS adapted to work over UDP. It provides the same security guarantees as TLS but handles the unreliability of UDP explicitly.

What DTLS adds on top of TLS

Retransmission timers — if a handshake message is not acknowledged, DTLS retransmits it. TLS doesn't need this because TCP guarantees delivery.

Message sequence numbers — DTLS numbers every handshake message so out-of-order delivery can be detected and handled.

Cookie exchange — before the full handshake begins, DTLS adds an extra round trip (HelloVerifyRequest) to verify the client's IP address. This prevents DoS attacks where an attacker floods the server with fake ClientHello messages from spoofed source IPs.

```
Client → Server: ClientHello
Server → Client: HelloVerifyRequest + Cookie ← UDP-specific, prevents DoS
Client → Server: ClientHello + Cookie
Server → Client: ServerHello + key material
Client → Server: Finished
Server → Client: Finished
→ Encrypted DTLS session begins
```

DTLS record structure

Every DTLS record (equivalent to a TCP segment) has:

DTLS header data		Record	seq num	Epoch	IV	Encrypted
13 bytes	MAC	8 bytes		2 bytes	16 bytes	variable
16 bytes						

Epoch — increments every time new session keys are established (e.g., after re-keying). Combined with the record sequence number, it makes every record uniquely identifiable for replay prevention.

DTLS versions

- **DTLS 1.0** — based on TLS 1.1, older, avoid
- **DTLS 1.2** — based on TLS 1.2, current standard, what SmartDB uses
- **DTLS 1.3** — based on TLS 1.3, newer, better security, limited mbedTLS support — flagged for future consideration

Pre-Shared Keys (PSK)

Standard TLS/DTLS uses certificates for authentication — a certificate authority (CA) signs a server's certificate, proving its identity to any client that trusts that CA. This is how HTTPS works.

For SmartDB, certificates are unnecessary complexity:

- We know exactly which devices exist (closed system)
- ESP32 nodes are not "strangers" to the gateway — they're our own hardware
- Certificate infrastructure (CA, signing, revocation) would add significant deployment complexity with no benefit

PSK (Pre-Shared Key) is the alternative — both sides share a secret key that is provisioned at manufacturing/flashing time. The DTLS handshake uses this shared secret to prove both parties know it, without ever transmitting the secret itself.

```
Provisioning time: PSK flashed onto ESP32 + stored in gateway config
Runtime handshake: both sides prove they know the PSK via a key derivation
                    process — the PSK itself never crosses the network
```

PSK in SmartDB

Each node has:

- A **PSK identity** — a string identifying the node (e.g., "node_001")
- A **PSK value** — a secret byte array (e.g., 16-32 random bytes)

The gateway looks up the PSK value using the PSK identity during the handshake. If the PSK doesn't match, the handshake fails — the node is rejected.

PSK security considerations

- If a PSK is compromised, that node's communications are compromised — but only that node (per-node PSKs limit blast radius)
- PSKs should be rotated periodically — this requires a CONFIG message carrying a new PSK (not yet defined in the protocol, flagged for v2)
- Store PSKs in ESP32's NVS (Non-Volatile Storage) with encryption enabled, not in plaintext flash

mbedtls

mbedtls (formerly PolarSSL) is an open-source TLS/DTLS library designed for embedded and constrained devices. It is:

- Included in ESP-IDF — no separate installation needed
- Written in C — runs on ESP32's Xtensa LX6 cores
- Configurable — you can enable only the cipher suites you need, saving flash
- Uses ESP32's hardware AES accelerator automatically

Key mbedtls components for SmartDB

Component	Purpose
<code>mbedtls_ssl_context</code>	Main DTLS session state
<code>mbedtls_ssl_config</code>	Configuration (cipher suite, PSK, transport type)

Component	Purpose
<code>MBEDTLS_ENTROPY_CONTEXT</code>	Entropy source for random number generation
<code>MBEDTLS_CTR_DRBG_CONTEXT</code>	DRBG — deterministic random byte generator
<code>MBEDTLS_TIMING_DELAY_CONTEXT</code>	DTLS retransmission timer

Minimal DTLS PSK client configuration

```
// 1. Set DTLS (datagram) transport – not stream (TLS)
mbedtls_ssl_config_defaults(&conf,
    MBEDTLS_SSL_IS_CLIENT,
    MBEDTLS_SSL_TRANSPORT_DATAGRAM, // ← this is what makes it DTLS not
    TLS
    MBEDTLS_SSL_PRESET_DEFAULT);

// 2. Set cipher suite – AES-128-CBC with PSK
int ciphersuites[] = {
    MBEDTLS_TLS_PSK_WITH_AES_128_CBC_SHA256,
    0
};
mbedtls_ssl_conf_ciphersuites(&conf, ciphersuites);

// 3. Load PSK
const unsigned char psk[] = { 0x1a, 0x2b, 0x3c, ... };
const char psk_id[] = "node_001";
mbedtls_ssl_conf_psk(&conf, psk, sizeof(psk),
    (const unsigned char*)psk_id, strlen(psk_id));

// 4. Set timer callbacks (required for DTLS retransmission)
mbedtls_ssl_set_timer_cb(&ssl,
    &timer,
    mbedtls_timing_set_delay,
    mbedtls_timing_get_delay);
```

Enabling hardware AES on ESP32

In your ESP-IDF project's `sdkconfig` (via `idf.py menuconfig`):

Component config → mbedTLS → Hardware acceleration → Enable hardware AES

Confirm it's enabled — software AES on ESP32 is ~3x slower and uses more CPU cycles, which affects your 100ms sampling duty cycle.

Replay attack prevention

What a replay attack is

An attacker captures a valid encrypted DTLS packet — for example, an OVERRIDE 0x06 command you sent legitimately. Later, they retransmit that exact captured packet. Without replay prevention, the receiver would see a valid, properly authenticated packet and execute the OVERRIDE command.

How SmartDB prevents it

Two layers of protection:

Layer 1 — DTLS record sequence numbers

DTLS numbers every record (epoch + sequence number). The receiver tracks the last accepted sequence number and rejects any record with a sequence number it has already seen or that is too far in the past.

Layer 2 — Application layer sequence numbers

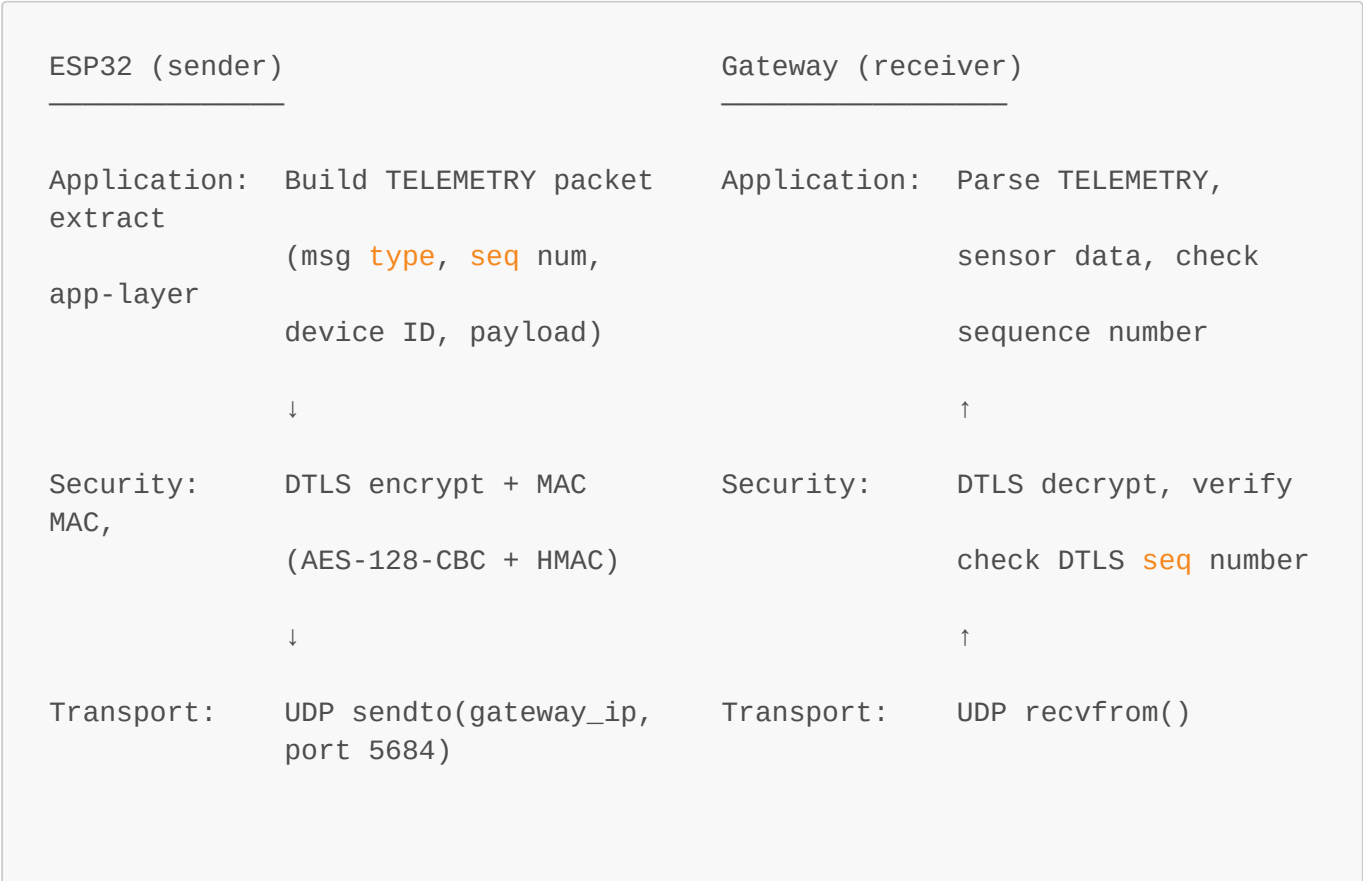
SmartDB's application protocol also includes a sequence number in every packet header. The receiver maintains a window of accepted sequence numbers (window size = 32). Any packet with a sequence number older than the window is unconditionally rejected — regardless of valid DTLS authentication.

This double protection means even if an attacker somehow bypassed the DTLS layer's sequence check, the application layer provides a second independent check.

Why two layers?

DTLS session state (including sequence numbers) resets on reconnection — after a node reboots and re-establishes DTLS, DTLS sequence numbers start over. An application-layer sequence number that persists independently adds protection across session boundaries.

How everything fits together in SmartDB





What an intercepted packet looks like to an attacker

Raw bytes on the network:

a3 f2 9c 41 0e 77 b2 ... (random-looking ciphertext)

Without the PSK:

- Cannot decrypt (AES-128 with 2^{128} key space)
- Cannot forge a valid MAC (HMAC-SHA256 requires the key)
- Cannot replay old packets (sequence number window)
- Cannot inject new packets (MAC verification will fail)

Sequence of events for a normal TELEMETRY message

1. ESP32 samples ADS1115 + MLX90640 (100ms cycle)
2. ESP32 computes Irms, THD, FFT features, anomaly score
3. ESP32 assembles TELEMETRY 0x01 packet with current sequence number
4. mbedTLS encrypts packet with AES-128-CBC, appends HMAC-SHA256 MAC
5. DTLS wraps in a record with its own sequence number + epoch
6. UDP sends to gateway IP, port 5684
7. Gateway receives UDP packet
8. DTLS verifies DTLS sequence number (replay check layer 1)
9. DTLS decrypts AES-128-CBC, verifies HMAC-SHA256
10. Application layer verifies app sequence number (replay check layer 2)
11. Application layer parses TELEMETRY payload, stores to Firebase

Glossary

Term	Meaning
AES	Advanced Encryption Standard — symmetric block cipher
CBC	Cipher Block Chaining — mode of operation for AES
Ciphertext	Encrypted data — unreadable without the key
CTR_DRBG	Counter mode Deterministic Random Byte Generator — used by mbedTLS for key/IV generation
DTLS	Datagram TLS — TLS adapted for UDP
Epoch	DTLS counter that increments on session rekeying
GCM	Galois/Counter Mode — alternative to CBC, combines encryption + authentication
HMAC	Hash-based Message Authentication Code

Term	Meaning
IV	Initialization Vector — random value used to start CBC chain
MAC	Message Authentication Code — proves integrity and authenticity
mbedTLS	Embedded TLS/DTLS library used on ESP32
NVS	Non-Volatile Storage — ESP32's flash storage for config data
Plaintext	Unencrypted readable data
PSK	Pre-Shared Key — secret shared between node and gateway at provisioning
PSK identity	String identifier for a node's PSK (e.g., "node_001")
Replay attack	Attacker retransmits a previously captured valid packet
Session key	Symmetric key derived during handshake, used for the session's AES encryption
TLS	Transport Layer Security — security protocol for TCP connections
UDP	User Datagram Protocol — connectionless transport layer protocol
XOR	Bitwise exclusive-or — fundamental operation in CBC chaining